

Programmation Concurrente en Multi-Threads

Matthieu PERRIN

Nantes Université

Laboratoire des Sciences du Numérique de Nantes

UMR CNRS 6004

Bureau 410, bâtiment 34

`matthieu.perrin@univ-nantes.fr`

Master en Informatique, première année

Parcours ALMA

2026-2027

Licence : CC BY-SA 4.0 (© ⓘ ⓘ) — Matthieu Perrin (*sauf mention contraire*)

Code source : <https://github.com/DistributedComputing>

Partie A. Du parallélisme à la concurrence

I. Généralités

1. Vue générale
2. Bibliographie

II. Programmation parallèle

III. Exécutions concurrentes

Positionnement dans le master

S1 : Programmation concurrente en multi-threads

- ▶ Programmation en mémoire partagée (multi-threads en Java)
- ▶ Synchronisation bloquante (verrous, moniteurs)
- ▶ Raisonnement sur la concurrence

S2 : Programmation distribuée

- ▶ Algorithmique en mémoire partagée
- ▶ Synchronisation non-bloquante (lock-free / wait-free)
- ▶ Limites à la calculabilité (impossibilité du consensus)

S3 : Services de communication et systèmes distribués

- ▶ Algorithmique en passages de messages
- ▶ Tolérance aux fautes (crash)
- ▶ Machine à états répliquée (Paxos)

Vue générale

“ *If you can get away with it, avoid using threads. Threads can be difficult to use, and they make programs harder to debug.*

Ancienne documentation de Java ”

But du cours

- ▶ Comprendre les problèmes inhérents aux systèmes répartis
- ▶ Apprendre à raisonner sur la concurrence
- ▶ Déjouer les fausses intuitions
- ▶ Être en mesure de lire les documentations officielles
- ▶ Mettre en œuvre les solutions les plus simples

Langages

- ▶ Principalement Java
- ▶ Généralisations bienvenues

Bibliographie

Sites Web

- ▶ docs.oracle.com/javase/tutorial/essential/concurrency/index.html
- ▶ <https://jenkov.com/tutorials/java-concurrency/index.html>

Ouvrages

- ▶ *The Art of Multiprocessor Programming*
Maurice Herlihy & Nir Shavit (Morgan Kaufmann 2008)
- ▶ *Concurrent Programming – Algorithms, Principles, and Foundations*
Michel Raynal (Springer 2013)
- ▶ *Concurrent Programming in Java : Design Principles and Patterns*
Doug Lea (Addison-Wesley 1996)

 Les articles et ouvrages sont sur Madoc.

 Exemples de code du cours : <https://github.com/DistributedComputing/Exemples-Java>.

Partie A. Du parallélisme à la concurrence

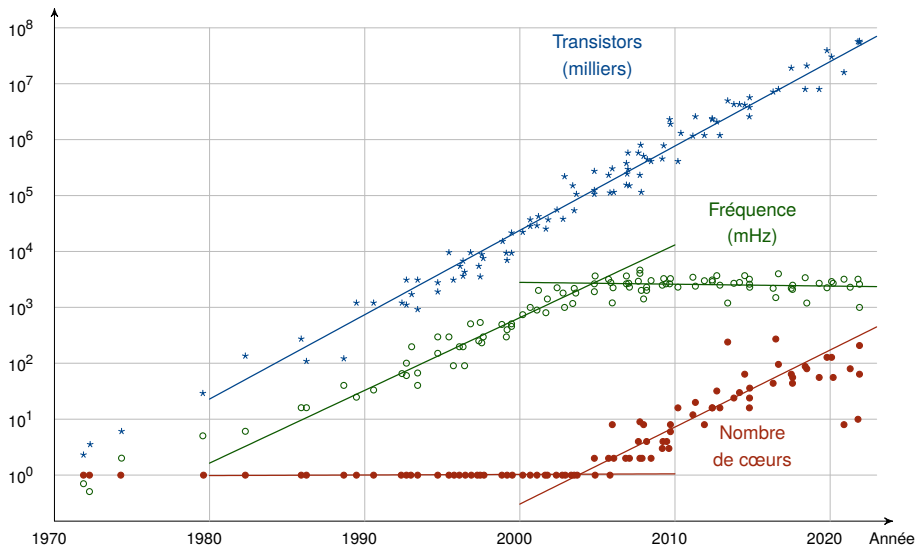
I. Généralités

II. Programmation parallèle

1. Motivations
2. Gestion des threads
3. Limites de la parallélisation
4. Introduction à la concurrence

III. Exécutions concurrentes

Fin de la loi de Moore ?

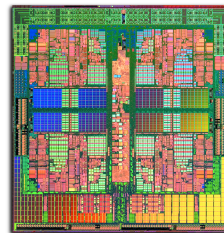
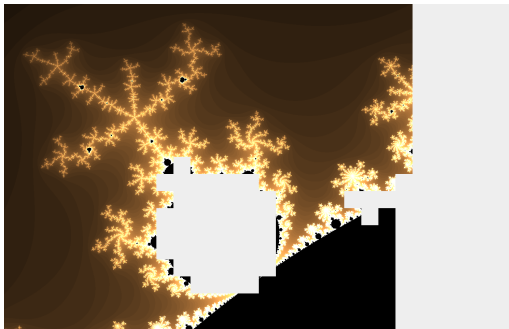


CC BY 4.0 – Karl Rupp 2022. *50 Years of Microprocessor Trend Data*. (Github) – présentation modifiée.
Données collectées par M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten et K. Rupp.

Buts de la programmation multi-cœurs

Calculs coûteux

- En temps de processeur (simulations scientifiques, ...)



Processeur¹

© 1. Advanced Micro Devices, Inc. (AMD) – usage autorisé avec attribution (Wikimedia)

Buts de la programmation multi-cœurs

Calculs coûteux

- ▶ En temps de processeur (simulations scientifiques, ...)
- ▶ En entrées/sorties (recherche dans les fichiers, ...)

```
$ java WebGrep Nantes https://fr.wikipedia.org/wiki/Nantes  
https://fr.wikipedia.org/wiki/Nantes  
https://fr.wikiquote.org/wiki/Nantes  
https://fr.wiktionary.org/wiki/Nantes  
https://fr.wikivoyage.org/wiki/Nantes  
...
```

Buts de la programmation multi-cœurs

Calculs coûteux

- ▶ En temps de processeur (simulations scientifiques, ...)
- ▶ En entrées/sorties (recherche dans les fichiers, ...)

Activités coopérantes

- ▶ Interface réactive (compilation à la volée, ...)
- ▶ Traitement de requêtes sur un serveur, ...

Stratégie de parallélisation

- ▶ Découper le problème en tâches indépendantes
- ▶ Associer des tâches à chaque cœur
- ▶ S'assurer que les cœurs collaborent correctement

Approches pour la programmation parallèle

“ A primary aim of an operating system is to share a computer installation among many programs making unpredictable demands upon its resources. ”

C.A.R Hoare

Processus versus thread

- ▶ **Informatique** = traitement automatique de l'**information**
- ▶ Le système d'exploitation abstrait le **processeur** et la **mémoire**
 - Processus : contexte d'exécution d'un programme (partage de mémoire)
 - Thread : description d'une exécution séquentielle (partage du processeur)
- ▶ Chaque processus contient un ou plusieurs threads

Approches pour la programmation parallèle

Programmation multi-processus

- ▶ Plusieurs processus, un thread par processus
- ▶ Communication par passage de messages
- ▶ Avantages :
 - Robustesse** : les processus sont isolés les uns des autres par l'OS
 - Répartition** : on peut répartir les processus sur un réseau de machines (passage à l'échelle)

Programmation multi-threads

- ▶ Un seul processus, plusieurs threads au sein du même processus
- ▶ Communication par mémoire partagée
- ▶ Avantages :
 - Efficacité** : données et références partagées (pas de copie)
 - Simplicité** : intégration presque transparente au sein des langages de programmation

Intégration aux langages de programmation

Java

La JVM gère le multithreading et crée des threads internes (GC, JIT...)

API historique Tâches, threads et outils basiques pour la synchronisation

Depuis Java 5 Bibliothèque `java.util.concurrent`¹

Depuis Java 21/24 (LTS) threads plateforme et virtuels (Project Loom)

C++

Donne accès aux threads du système.

Avant C++11 accès direct aux threads plateforme (**POSIX** et **WINAPI**).

Depuis C++11 intégration d'une API à la bibliothèque standard.

1. repose en grande partie sur les travaux de Doug Lea (<http://gee.cs.oswego.edu/dl/index.html>)

Tâche

- ▶ Code exécuté par un thread.

C++

N'importe quelle entité callable (fonction, foncteur, ...)

```
void printer (string message) {  
    cout << message << endl;  
}
```

Java

Implémentation de l'interface fonctionnelle `Runnable`

```
public record Printer (String message) implements Runnable {  
  
    public void run () {  
        System.out.println(message);  
    }  
  
}
```

Thread et contexte d'exécution

Java : classe `java.lang.Thread`

```
var task = new Printer("hello world");  
// Runnable task = () -> System.out.println("hello world");  
var thread = new Thread(task);  
thread.start();  
// Thread t = Thread.startVirtualThread(task);
```

C++ : classe `std::thread`

```
thread t(printer, "hello world");  
// Started by constructor
```

Exemple

```
var messages = new String[]{"Hello", "Multithreaded", "World"};  
for(var message : messages) {  
    new Thread(()->System.out.print(message)).start();  
}
```

 Exemples-Java/snippets.introduction.HelloWorld

Principales méthodes de la classe gérant les threads

Obtenir le thread qui contrôle la tâche en cours

Java : `Thread.currentThread()`

C++ : `std::this_thread::get_id()`

Principales méthodes de la classe gérant les threads

Obtenir le thread qui contrôle la tâche en cours

Java : `Thread.currentThread()`

C++ : `std::this_thread::get_id()`

Endormir le thread courant pour $_{ms}$ millisecondes

Java : `Thread.sleep(ms)`

C++ : `std::this_thread::sleep_for(std::chrono::milliseconds(ms));`

Principales méthodes de la classe gérant les threads

Obtenir le thread qui contrôle la tâche en cours

Java : `Thread.currentThread()`

C++ : `std::this_thread::get_id()`

Endormir le thread courant pour ms millisecondes

Java : `Thread.sleep(ms)`

C++ : `std::this_thread::sleep_for(std::chrono::milliseconds(ms));`

Attente de la terminaison d'un thread t

Java ou C++ : `t.join()`

attention : en C++, un thread doit être détaché avant la destruction de l'objet t (`t.join()` ou `t.detach()`).

Interrompt proprement un thread

Forcer un thread à s'arrêter

Impossible : `public void stop()` est dépréciée !

Méthode `public void interrupt()`

Demande la terminaison.

- ▶ Lève un drapeau booléen pour avertir le thread.
- ▶ La tâche peut (doit) vérifier régulièrement les demandes d'interruption
 - ▶ `public static boolean interrupted()` rebaisse le drapeau
 - ▶ `public boolean isInterrupted()` ne rebaisse pas le drapeau
- ▶ Si le thread est bloqué ou en attente, `InterruptedException` est lancée.

Équivalent en C++

Avant C++20 : Utiliser une variable booléenne **atomique** comme drapeau.

Depuis C++20 : `std::jthread::request_stop()` + `std::jthread::get_stop_token()`

L'interface `ExecutorService`

Association tâche / contexte d'exécution

`java.util.concurrent` offre les outils pour construire simplement des associations $n \leftrightarrow m$ avec différentes stratégies

```
public interface ExecutorService {  
    void execute (Runnable task);  
    Future<?> submit (Runnable task);  
    Future<T> submit (Callable<T> task);  
    List<Future<T>> invokeAll (Collection<Callable<T>> tasks)  
        throws InterruptedException;  
    T invokeAny (Collection<Callable<T>> tasks)  
        throws InterruptedException, ExecutionException;  
}
```

Exemple

```
var threadPool = Executors.newFixedThreadPool(5);  
for(int i = 0; i < 50; ++i){  
    int k = i;  
    threadPool.execute(() -> System.out.println(k));  
}  
threadPool.shutdown();  
threadPool.awaitTermination(1, TimeUnit.MINUTES);
```

Tâche retournant un résultat

L'interface *Callable*

Version générique d'une tâche qui retourne un résultat.

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```

L'interface *Future*

Version générique du résultat d'un calcul *asynchrone*.

```
public interface Future<V> {  
    boolean cancel( boolean mayInterruptIfRunning );  
    V get() throws InterruptedException, ExecutionException;  
    V get( long timeout, TimeUnit unit ) throws...;  
    boolean isCancelled();  
    boolean isDone();  
}
```

Implémentations de `ExecutorService`

Classe `java.util.concurrent.Executors`

Factory qui permet d'instancier simplement des `ExecutorService`, dont :

- ▶ `ThreadPoolExecutor` : implante différentes variantes basées sur le concept de *pool* de `Thread` : *single*, *fixed*, *cached*
- ▶ `ScheduledThreadPoolExecutor` : stratégie permettant de préciser un délai lors de la soumission, à écouler entre la soumission et le début de l'exécution de la tâche
- ▶ `ForkJoinPool` : stratégie particulière pour exécuter des `ForkJoinTask`, qui créent elles-mêmes des tâches
- ▶ `VirtualThreadPerTaskExecutor` : "green threads" gérés par le langage et non par le système

Quelle stratégie est la plus efficace ?

Ça dépend de l'application !

- ▶ Trop peu de threads et on n'exploite pas les capacités de calcul.
- ▶ Trop de threads et on perd du temps à leur manipulation.

Ça dépend de la plate-forme !

- ▶ Machine : nombre de cœurs, hiérarchie mémoire, etc.
- ▶ OS : politique d'ordonnancement des threads côté noyau.
- ▶ JVM : implémentation des stratégies, politique d'ordonnancement des threads côté utilisateur.

Mesurer les performances !

Limites de la parallélisation



Gene M. Amdahl

CC BY 3.0-unported – Perry Kivolowitz 2008 (Wikimedia)

G. M. Amdahl. *Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities*. AFIPS (1967)

Limites de la parallélisation

Question

- ▶ Un sondeur a besoin de 22 jours pour interroger 1000 sondés.
- ▶ Combien de temps faut-il à 22 sondeurs pour faire le même sondage ?

Limites de la parallélisation

Question

- ▶ Un sondeur a besoin de 22 jours pour interroger 1000 sondés.
- ▶ Combien de temps faut-il à 22 sondeurs pour faire le même sondage ?

Question

- ▶ Une éléphante a besoin de 22 mois pour donner la vie.
- ▶ Combien de temps faut-il à 22 éléphantess pour faire un éléphanteau ?

Limites de la parallélisation

Question

- ▶ Un sondeur a besoin de 22 jours pour interroger 1000 sondés.
- ▶ Combien de temps faut-il à 22 sondeurs pour faire le même sondage ?

Question

- ▶ Une éléphante a besoin de 22 mois pour donner la vie.
- ▶ Combien de temps faut-il à 22 éléphantes pour faire un éléphanteau ?

Cas général

- ▶ Une proportion p du temps d'exécution peut être parallélisée.
- ▶ Une proportion $1 - p$ ne le peut pas.

La loi d'Amdahl

Définition – Accélération

p : Proportion du temps d'exécution parallélisable

$T(n)$: Temps nécessaire à n threads

$S(n)$: Accélération (Speedup) pour n threads $S(n) = \frac{T(1)}{T(n)}$

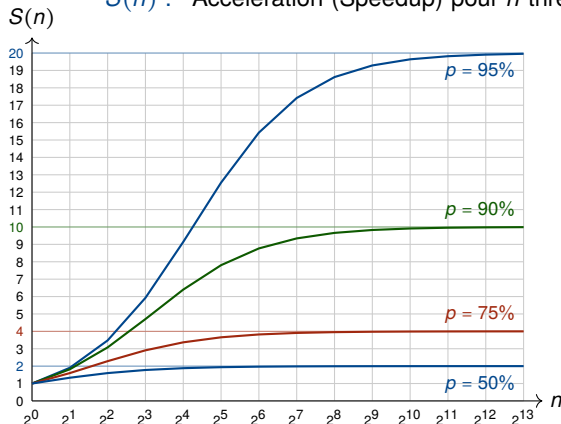
La loi d'Amdahl

Définition – Accélération

p : Proportion du temps d'exécution parallélisable

$T(n)$: Temps nécessaire à n threads

$S(n)$: Accélération (Speedup) pour n threads $S(n) = \frac{T(1)}{T(n)}$



$$S(n) = \frac{1}{1 - p + \frac{p}{n}}$$

$$S(n) < \frac{1}{1 - p}$$

$$\lim_{n \rightarrow \infty} S(n) = \frac{1}{1 - p}$$

Limites de la loi d'Amdahl

Hypothèses cachées :

- ▶ Parallélisation imparfaite
 - ▶ p dépend des données (loi de Gustafson)
 - ▶ p dépend de l'algorithme

$$S_p(n) \leq n$$

- ▶ Synchronisation non-parallélisable
 - ▶ Gestion de la concurrence

$$S_c(n) \leq 1$$

Formule plus réaliste

$$S(n) = \frac{1}{\frac{1-p}{S_c(n)} + \frac{p}{S_p(n)}} \leq \frac{1}{1-p + \frac{p}{n}}$$

Mesurez vos performances !

Problématique du cours

“ For many of the applications you may wish to parallelize, you will find that there are significant parts that can easily be determined as executable in parallel because they do not require any form of coordination or communication. However, (...) *there is no cookbook recipe for identifying these parts*. This is where the application designer needs to use his or her accumulated understanding of the algorithm being parallelized. Luckily, in many cases it is obvious how to find such parts.

The more substantial problem (...) is how to deal with the remaining parts of the program. As noted earlier, these are the parts that cannot be easily parallelized because the program must access shared data and requires interprocess coordination and communication in an essential way.

”

Maurice Herlihy & Nir Shavit

Programmation Multi-Threads : Théorie vs. Pratique

Théorie : parallélisme



Pratique : concurrence



Parallélisme versus concurrence

Principale difficulté

Parallélisme : Identifier les tâches indépendantes

Concurrence : Gérer l'incertitude

“ *A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.*

Leslie Lamport ”

 M. Raynal. *Parallel Computing vs. Distributed Computing : A Great Confusion ? (Position Paper)*. Euro-Par Workshops (2015)

Parallélisme versus concurrence

Principale difficulté

Parallélisme : Identifier les tâches indépendantes

Concurrence : Gérer l'incertitude

“ *A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.*

Leslie Lamport ”

Contexte d'exécution

Parallélisme : Choisi en fonction du problème à résoudre

Concurrence : Imposé *a priori* par le problème et l'environnement

 M. Raynal. *Parallel Computing vs. Distributed Computing : A Great Confusion ? (Position Paper)*. Euro-Par Workshops (2015)

Parallélisme versus concurrence

Principale difficulté

Parallélisme : Identifier les tâches indépendantes

Concurrence : Gérer l'incertitude

“ *A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.*

Leslie Lamport ”

Contexte d'exécution

Parallélisme : Choisi en fonction du problème à résoudre

Concurrence : Imposé *a priori* par le problème et l'environnement

Passage à l'échelle

Parallélisme : $S_p(n) = \Omega(n)$

Concurrence : $S_c(n) = \Omega(1)$

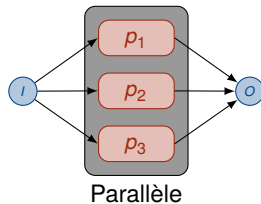
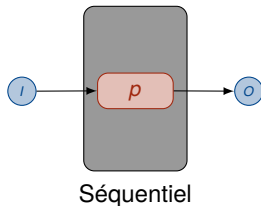
 M. Raynal. *Parallel Computing vs. Distributed Computing : A Great Confusion ? (Position Paper)*. Euro-Par Workshops (2015)

Point vocabulaire

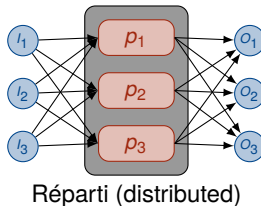
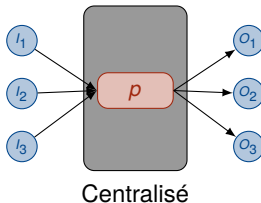
Une seule
unité d'exécution

Plusieurs
unités d'exécution

Problème
séquentiel



Problème
de
concurrence



Exercice

Problème de parallélisme ou de concurrence ?

1. Éviter les collisions sur la route.
2. Optimiser la vitesse de production d'une usine (Fordisme).
3. Élire le président de la république.
4. Massively multiplayer online role-playing game.
5. Prévoir la météo.
6. Les projets semestriels en groupe.
7. Un orchestre symphonique.

Partie A. Du parallélisme à la concurrence

I. Généralités

II. Programmation parallèle

III. Exécutions concurrentes

1. Notion d'asynchronisme
2. Relation happens-before de Lamport
3. Cohérence séquentielle
4. Machine à états d'un programme

Un premier exemple

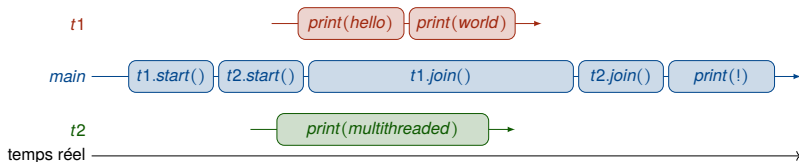
```
public static void main (String[] args) throws InterruptedException {  
    var t1 = new Thread(() -> {  
        System.out.print("Hello ");  
        System.out.print("World ");  
    });  
    var t2 = new Thread(() -> {  
        System.out.print("Multithreaded ");  
    });  
  
    t1.start();  
    t2.start();  
  
    t1.join();  
    t2.join();  
  
    System.out.print("!");  
}
```

Expérience

- ▶ Lancer le programme plusieurs fois. Qu'observe-t-on ?

Non déterminisme des sorties du programme

Une exécution possible



Asynchronisme

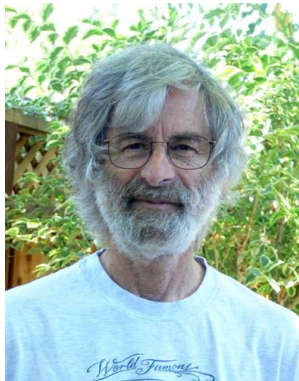
Il est impossible de prévoir la vitesse d'exécution :

- ▶ d'un thread par rapport à un autre,
- ▶ d'une portion de code exécutée par un thread par rapport à une autre,
- ▶ d'une exécution d'un thread par rapport à une autre.

Question

- ▶ Un programme n'est correct que si toutes ses exécutions le sont.
- ▶ Comment raisonner sur les exécutions concurrentes ?

Causalité et concurrence



Leslie Lamport¹

1. Prix Turing 2013 pour ses apports au calcul réparti

© CC-0 – usage autorisé sans restriction (Wikimedia)

L. Lamport. *Time, clocks, and the ordering of events in a distributed system*. CACM (1978)

L. Lamport. *How to make a multiproc. computer that correctly executes multiprocess programs*. IEEE ToC (1979)

La relation “happened before” (ordre causal)

“ The *concept of time* is fundamental to our way of thinking. *It is derived from the more basic concept of the order in which events occur.* (...) However, we will see that this concept must be carefully reexamined when considering events in a distributed system. (...) In a distributed system, it is sometimes impossible to say that one of two events occurred first. *The relation "happened before" is therefore only a partial ordering of the events in the system.* ”

Leslie Lamport

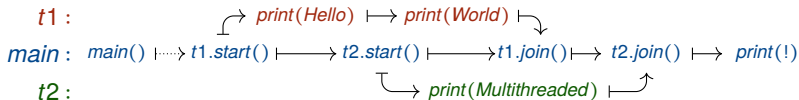
Définitions

- ▶ $e \mapsto e'$ si l'une des trois conditions s'applique :
 - ▶ **Ordre de programme** : le même thread produit e puis e'
 - ▶ **Précédence sémantique** : (précisé plus tard)
 - ▶ Par exemple, $e = t.start()$ et e' produit par t
 - ▶ Par exemple, e produit par t et $e' = t.join()$
 - ▶ **Transitivité** : $\exists e'', e \mapsto e'' \mapsto e'$
- ▶ e et e' sont **concurrents** ($e \parallel e'$) si $e \not\mapsto e'$ et $e' \not\mapsto e$.

Attention : $\mapsto$ est seulement un ordre **partiel** !

 L. Lamport. *Time, clocks, and the ordering of events in a distributed system*. CACM (1978)

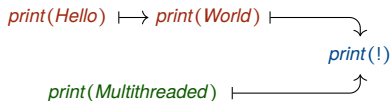
Exemple



Observations

- ▶ L'écriture "Hello" se produit avant l'écriture "World"
- ▶ Les deux écritures de *t1* sont concurrentes à l'écriture de *t2*
- ▶ Les trois se produisent avant l'écriture "!"

Projection sur l'objet *System.out*



Cohérence séquentielle

Il faudrait que...

“ (...) the result of any execution is *the same as* if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program. ”

Leslie Lamport

Entrelacement

Un **entrelacement** d'un ordre partiel $\leq$ est un ordre total qui contient $\leq$.

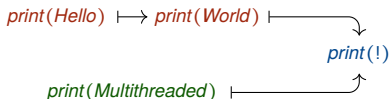
Cohérence séquentielle

Une exécution est **séquentiellement cohérente** si son **résultat observable** est **le même que** celui d'un **entrelacement** de son ordre “happened before”.

 L. Lamport. *How to make a multiproc. computer that correctly executes multiprocess programs*. IEEE ToC (1979)

Exemple

Relation happened before



Entrelacements possibles

- ▶ `print(Hello); print(World); print(Multithreaded); print(!);`
Résultat : Hello World Multithreaded !
- ▶ `print(Hello); print(Multithreaded); print(World); print(!);`
Résultat : Hello Multithreaded World !
- ▶ `print(Multithreaded); print(Hello); print(World); print(!);`
Résultat : Multithreaded Hello World !

Machine à états d'un programme

Si l'exécution est **séquentiellement cohérente**, on peut utiliser une **sémantique d'entrelacement** : l'ensemble des exécutions est décrit par une **machine à états non-déterministe**.

États du système

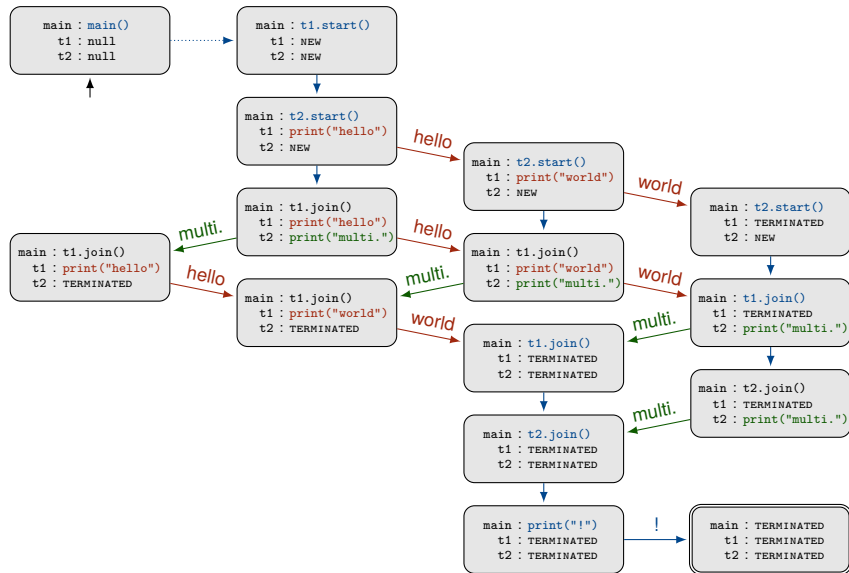
À chaque instant, la **configuration globale** est définie par :

- ▶ Un ensemble de threads en cours d'exécution
- ▶ L'état local de chaque thread
 - Pile d'appels** : valeur de ses variables locales
 - Avancement** : prochaine instruction à exécuter
 - Éligibilité** : peut-il exécuter sa prochaine instruction ?
- ▶ L'état de la mémoire
 - ▶ ensemble de variables partagées
 - ▶ valeur de chacune d'entre elles

Transitions

Un **ordonnanceur** décide quel thread exécutera sa prochaine instruction.

Machine à états de l'exemple



Mise en garde

“ A large fraction of the flaws in software development are due to programmers not fully understanding all the possible states their code may execute in.

In a multithreaded environment, the lack of understanding and the resulting problems are greatly amplified, almost to the point of panic if you are paying attention. ”

John Carmack

Limitation des états mutables partagés

- ▶ Identifier les objets partagés
 - ▶ En minimiser l'utilisation
 - ▶ S'assurer qu'ils sont *thread-safe*
- ▶ Utiliser des structures de données immuables
 - ▶ Éliminer les accesseurs
 - ▶ Déclarer les champs **final**
- ▶ Favoriser les fonctions pures

Partie B. Synchronisation bloquante

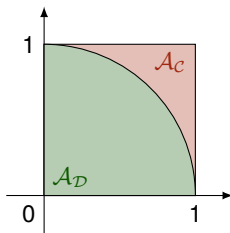
IV. Synchronisation par verrous

1. Section critique
2. Utilisation des verrous

V. Moniteurs

Exemple de programme parallèle

Calcul de π par la méthode de Monte-Carlo



$$\mathcal{A}_C = 1$$

$$\mathcal{A}_D = \frac{\pi}{4}$$

$$\pi = 4 \times \mathbb{P}(x^2 + y^2 < 1 \mid \langle x, y \rangle \in [0; 1]^2)$$

Expérience

- ▶ Tirer `nbIterations` points dans l'intervalle $[0; 1]^2$
- ▶ Paralléliser entre `nbThreads` threads
- ▶ Mesurer le temps d'exécution en fonction de `nbIterations` et `nbThreads`

 `Exemples-Java/snippets.blocking.Pi1_Sequential`

 `Exemples-Java/snippets.blocking.Pi2_Parallel`

Expérience : 8 threads tirent 10 000 000 pièces

Expérience 1 :

```
var random = new Random();  
for(int i = 0; i < 8; i++) {  
    new Thread(() -> {  
        for(int j = 0; j < 10000000; j++) random.nextBoolean();  
    }).start();  
}
```

Expérience 2 :

```
for(int i = 0; i < 8; i++) {  
    new Thread(() -> {  
        var random = new Random();  
        for(int j = 0; j < 10000000; j++) random.nextBoolean();  
    }).start();  
}
```

Expérience 3 :

```
for(int i = 0; i < 8; i++) {  
    new Thread(() -> {  
        var random = ThreadLocalRandom.current();  
        for(int j = 0; j < 10000000; j++) random.nextBoolean();  
    }).start();  
}
```

Expérience : 8 threads tirent 10 000 000 pièces

Expérience 1 : 7736ms

```
var random = new Random();  
for(int i = 0; i < 8; i++) {  
    new Thread() -> {  
        for(int j = 0; j < 10000000; j++) random.nextBoolean();  
    }.start();  
}
```

Expérience 2 : 126ms

```
for(int i = 0; i < 8; i++) {  
    new Thread() -> {  
        var random = new Random();  
        for(int j = 0; j < 10000000; j++) random.nextBoolean();  
    }.start();  
}
```

Expérience 3 : 17ms

```
for(int i = 0; i < 8; i++) {  
    new Thread() -> {  
        var random = ThreadLocalRandom.current();  
        for(int j = 0; j < 10000000; j++) random.nextBoolean();  
    }.start();  
}
```

Tirer des nombres aléatoires en Java

“ Instances of `java.util.Random` are threadsafe. However, the concurrent use of the same `java.util.Random` instance across threads may encounter contention and consequent poor performance. Consider instead using `ThreadLocalRandom` in multithreaded designs.

Javadoc de `java.util.Random` ”

Définition – thread safety

Un objet est **thread safe** s'il a été conçu pour s'exécuter dans un environnement concurrent

- ▶ Pas de spécification formelle
- ▶ Tout objet partagé entre plusieurs threads doit être thread-safe
- ▶ Tout objet immutable est thread-safe
- ▶ Tout objet mutable thread-safe a des coûts liés à la synchronisation

Utilisation de ThreadLocalRandom

```
private static int result = 0;

public void pi(int nbThreads, int nbIterations) {
    threads = new Thread[n];
    for(int i = 0; i<nbThreads; i++) {
        threads[i] = new Thread(() -> {
            var random = ThreadLocalRandom.current();
            for(int j = 0; j<nbIterations; j++) {
                double x = random.nextDouble();
                double y = random.nextDouble();
                if(x*x + y*y < 1) result++;
            }
        });
    }

    for(var t : threads) t.start();
    for(var t : threads) t.join();

    System.out.println(result);
}
```

Explications

Code simplifié

```
class Task implements Runnable {  
    static int result = 0;  
    public void run() {  
        result++;  
    }  
}
```

On lance deux fois la tâche en parallèle, que devrait valoir `result` à la fin ?

Explications

Code simplifié

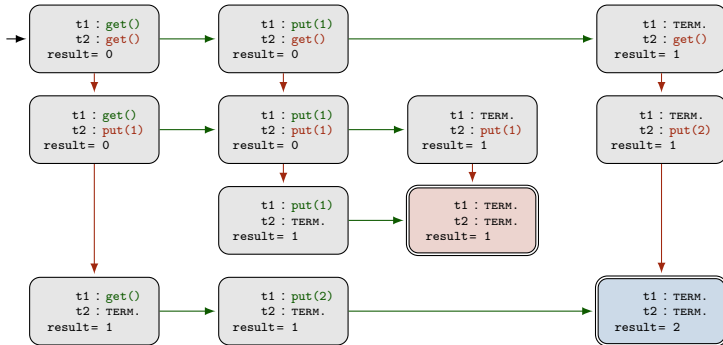
```
class Task implements Runnable {  
    static int result = 0;  
    public void run() {  
        result++;  
    }  
}
```

On lance deux fois la tâche en parallèle, que devrait valoir `result` à la fin ?

Code décompilé : `javap -c Task.class`

```
public void run();  
  Code:  
    0: getstatic    #2      // Field result:I  
    3: iconst_1  
    4: iadd  
    5: putstatic    #2      // Field result:I  
    8: return
```

Ce que l'on a



Quelques entrelacements possibles

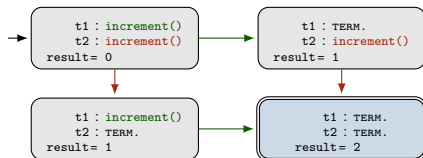
- ▶ $t_1 : \text{get}(); t_1 : \text{put}(1); t_2 : \text{get}(); t_2 : \text{put}(2);$ $\text{result} = 2$
- ▶ $t_1 : \text{get}(); t_2 : \text{get}(); t_1 : \text{put}(1); t_2 : \text{put}(1);$ $\text{result} = 1$

Ce que l'on voudrait

Rappel : exécution séquentiellement cohérente

Le résultat observable d'une exécution est le même que celui d'un entrelacement de l'ordre "happened before"

```
interface Counter {
    void increment() ;
    int get() ;
}
```



Entrelacements possibles

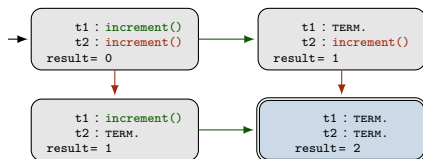
- ▶ $t_1 : \text{increment}(); t_2 : \text{increment}();$ $\text{result} = 2$
- ▶ $t_2 : \text{increment}(); t_1 : \text{increment}();$ $\text{result} = 2$

Ce que l'on voudrait

Rappel : exécution séquentiellement cohérente

Le résultat observable d'une exécution est le même que celui d'un entrelacement de l'ordre "happened before"

```
interface Counter {
    void increment() ;
    int get() ;
}
```



Entrelacements possibles

- ▶ t_1 : *increment()*; t_2 : *increment()*; *result* = 2
- ▶ t_2 : *increment()*; t_1 : *increment()*; *result* = 2

Objet partagé séquentiellement cohérent

Le résultat observable des opérations sur l'objet partagé d'une exécution est le même que celui d'un entrelacement de l'ordre "happened before"

Verrou

java.util.concurrent.locks.Lock

Objet informatique qui ne peut être possédé que par un thread à la fois.

```
interface Lock {
    void lock()
    void unlock()
    void lockInterruptibly() throws InterruptedException
    boolean tryLock()
    ...
}
```

Exclusion mutuelle

Java

```
int value = 0;
Lock lock = new ReentrantLock();

void increment() {
    lock.lock();
    try{ ++value; }
    finally{lock.unlock();}
}
```

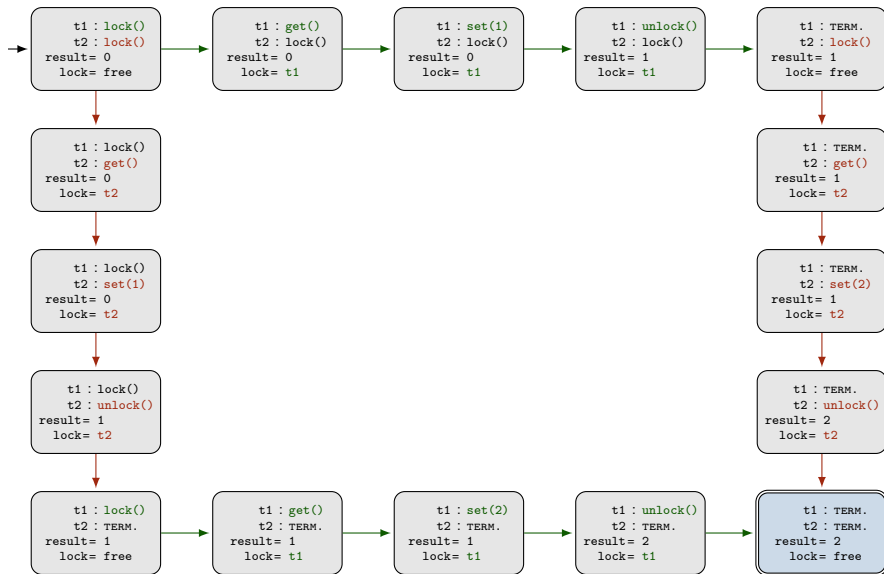
Section critique

C++

```
int value = 0;
mutex lock;

void increment() {
    lock_guard<mutex> g(lock);
    ++value;
}
```

Incrémentations concurrentes



Point vocabulaire

Section critique

Séquence d'instructions qui manipule des données partagées.

Exclusion mutuelle

Prédicat logique :

« à tout instant, il y a au plus un thread en section critique ».

Verrou

Objet informatique qui ne peut être possédé que par un thread à la fois.

Attention

- ▶ Un **verrou** (*objet informatique*) protège une **section critique** (*portion de code*) pour en assurer l'**exclusion mutuelle** (*propriété logique*).
- ▶ Attention au mot "Mutex"

Origine : contraction de "mutual exclusion"

En C++ : synonyme de "verrou"

Mot-clé *synchronized* en Java

Moniteur d'exécution en Java

Un **verrou** est associé à chaque :

- ▶ objet instancié dans la JVM,
- ▶ classe.

Mot-clé *synchronized*

- ▶ Méthode déclarée **synchronized**
 - ▶ le verrou sur **this** est pris pour la durée de l'exécution de la méthode.
- ▶ Bloc **synchronized{}**
 - ▶ le verrou est pris pour la durée de l'exécution du bloc.

Mot-clé *synchronized* en Java

Moniteur d'exécution en Java

Un **verrou** est associé à chaque :

- ▶ objet instancié dans la JVM,
- ▶ classe.

Mot-clé *synchronized*

- ▶ Méthode déclarée **synchronized**
 - ▶ le verrou sur **this** est pris pour la durée de l'exécution de la méthode.
- ▶ Bloc **synchronized{}**
 - ▶ le verrou est pris pour la durée de l'exécution du bloc.

Avantages

- ▶ Simple d'utilisation.
- ▶ Mise en œuvre efficace.

Inconvénients

- ▶ Peu flexible dans l'utilisation.
- ▶ Une seule sorte de verrou

Exemple

Synchronisation sur membre privé

```
class Counter {  
  
    private int value = 0;  
    private Object lock = new Object();  
  
    void increment() {  
        synchronized(lock) {  
            ++value;  
        }  
    }  
  
    int get() {  
        synchronized(lock) {  
            return value;  
        }  
    }  
  
}
```

Synchronisation sur `this`

```
class Counter {  
  
    private int value = 0;  
  
    void increment() {  
        synchronized(this) {  
            ++value;  
        }  
    }  
  
    int synchronized get() {  
        return value;  
    }  
  
}
```

-
-  Exemples-Java/snippets.blocking.Pi4_Synchronized
 -  Exemples-Java/snippets.blocking.Pi5_Independant
 -  Exemples-Java/snippets.blocking.Pi6_Collect

Grain de verrouillage

Verrouillage à gros grains

- ▶ Un seul verrou pour une structure de données
- ▶ Chaque méthode est entièrement protégée par le verrou
- ▶ Avantage : très simple à mettre en place
- ▶ Inconvénient : très inefficace

Verrouillage à grains fins

- ▶ Plusieurs verrous pour une structure de données
- ▶ Chaque verrou protège une partie des données
- ▶ Attention à quels verrous doivent être pris

Verrous à lectures et écritures

En java (*java.util.concurrent*)

```
ReadWriteLock lock = new ReentrantReadWriteLock();
```

► Lock writeLock = lock.writeLock();

1. Récupère le verrou **exclusif** pour protéger les écritures

► Lock readLock = lock.readLock();

1. Récupère le verrou **non exclusif** pour protéger les lectures

En C++ (*boost::thread* avant C++ 17, *std* depuis C++ 17)

```
shared_mutex lock;
```

► unique_lock<shared_mutex> writeLock(lock);

1. Crée un verrou **exclusif** pour protéger les écritures

► shared_lock<shared_mutex> readLock(lock);

1. Crée un verrou **non exclusif** pour protéger les lectures

Exemple

```
class Compteur {  
  
    private int v = 0;  
    private ReadWriteLock lock = new ReentrantReadWriteLock();  
  
    public void increment() {  
        lock.writeLock().lock();  
        try { ++v; }  
        finally { lock.writeLock().unlock(); }  
    }  
  
    public int get() {  
        lock.readLock().lock();  
        try { return v; }  
        finally { lock.readLock().unlock(); }  
    }  
}
```

Thread quittant du code en exclusion mutuelle

Quand un thread quitte une exclusion mutuelle

Deux cas de figure pour le verrou de l'objet :

- ▶ L'un des threads bloqués peut l'acquérir (ordre indéterminé).
- ▶ Il est libéré si aucun thread n'est bloqué.

Avant la terminaison d'un Thread

Il faut :

- ▶ libérer tous les verrous en sa possession.
- ▶ être certain qu'il laisse les données partagées dans un état cohérent.
- ▶ être certain qu'aucune autre tâche n'attend une action de sa part.

Partie B. Synchronisation bloquante

IV. Synchronisation par verrous

V. Moniteurs

1. Attente et signalement
2. Notion de moniteur
3. Utilisation du moniteur `Object`

Attente et signalement

```
class Counter {  
    private int value=0;  
  
    public synchronized void increment() {  
        ++value;  
    }  
  
    public synchronized int get() {  
        return value;  
    }  
  
    public static void main(String[] args) {  
        var counter = new Counter();  
  
        var tReader = new Thread(() -> {  
            while(counter.get()==0);  
            System.out.println("done");  
        });  
  
        var tWriter = new Thread( counter::increment);  
  
        tReader.start();  
        Thread.sleep(250);  
        tWriter.start();  
    }  
}
```

Attente et signalement

```
class Counter {  
  
    private int value=0;  
  
    public synchronized void increment() {  
        ++value;  
    }  
  
    public synchronized int get() {  
        return value;  
    }  
  
    public static void main(String[] args) {  
        var counter = new Counter();  
  
        var tReader = new Thread(() -> {  
            while(counter.get()==0);  
            System.out.println("done");  
        });  
  
        var tWriter = new Thread( counter::increment);  
  
        tReader.start();  
        Thread.sleep(250);  
        tWriter.start();  
    }  
}
```

Comment supprimer l'attente active ?

Sémaphore

Classe `java.util.concurrent.Semaphore`

- ▶ `new Semaphore(n)`
 1. Initialise un sémaphore avec n jetons
- ▶ `public void acquire() throws InterruptedException`
 1. S'il reste un jeton, donner un jeton.
 2. Sinon, attendre qu'un jeton se libère.
- ▶ `public void release()`
 1. Libérer un jeton.



Edsger W. Dijkstra

 CC-BY-SA-3.0-unported – Hamilton Richards 2002 (Wikimedia)

 E. W. Dijkstra. *Over de sequentialiteit van procesbeschrijvingen (About the sequentiality of process descriptions)* (1962-1963)

Signalement par sémaphore

```
public static void main(String[] args) {  
    var semaphore = new Semaphore(0);  
  
    new Thread(() -> {  
        semaphore.acquire();  
        System.out.println("done");  
    }).start();  
  
    Thread.sleep(250);  
  
    new Thread(semaphore::release).start();  
}
```

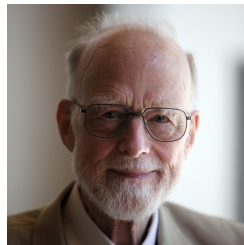
Moniteur

Construction de synchronisation

- ▶ Un moniteur est un objet encapsulant des données partagées.
- ▶ Accès aux données partagées en exclusion mutuelle.
- ▶ Méthodes gardées par des conditions bloquantes (gardes).



Per Brinch Hansen¹



Tony Hoare²

© 1. Per Brinch Hansen 1999 – usage autorisé avec attribution (Wikimedia)

© 2. CC-BY-SA-2.0-FR – Rama 2011 (Wikimedia)

1 P. B. Hansen. *Operating System Principles*. Prentice Hall (1973)

2 C. A. R. Hoare. *Monitors : an operating system structuring concept*. CACM (1974)

File bloquante

Interface `java.util.concurrent.BlockingQueue`

	Exception	Valeur spéciale	Bloquante	Timeout
Insérer	<code>add(e)</code>	<code>offer(e)</code>	<code>put(e)</code>	<code>offer(e, time, unit)</code>
Supprimer	<code>remove()</code>	<code>poll()</code>	<code>take()</code>	<code>poll(time, unit)</code>
Regarder	<code>element()</code>	<code>peek()</code>		

Le problème des producteurs et des consommateurs

```
BlockingQueue<Integer> queue = new LinkedBlockingQueue<Integer>();
```

```
Producer = new Thread(() -> {  
    int value = 17;  
    while(value != 1) {  
        queue.put(value);  
        if(value%2 == 0) value = value/2;  
        else value = value*3+1;  
        Thread.sleep(500);  
    }  
    queue.put(value);  
});
```

```
Consumer = new Thread(() -> {  
    int value = 1;  
    do {  
        value = queue.take();  
        System.out.println(value);  
    } while(value != 1);  
});
```

```
Producer.start();  
Consumer.start();
```

Barrières de synchronisation

Classe `java.util.concurrent.CountDownLatch`

- ▶ `public void await() throws InterruptedException`
 - ▶ Se met en attente jusqu'à ce que le compteur atteigne 0
- ▶ `public void countDown()`
 - ▶ Décrémente le compteur

Classe `java.util.concurrent.CyclicBarrier`

- ▶ `public int await() throws InterruptedException, BrokenBarrierExc.`
 - ▶ Se met en attente jusqu'à ce que tous les threads aient rejoint la barrière
 - ▶ Quand tous les threads ont rejoint la barrière, les threads sont réveillés et la barrière est réinitialisée.

Le moniteur universel en Java (*java.lang.Object*)

```
public final void java.lang.Object.wait() throws InterruptedException
```

- ▶ Met le thread en attente sur l'objet verrouillé.
- ▶ Il doit **posséder** le verrou de l'objet sur lequel il se met en attente.
- ▶ Lorsqu'il entre en attente, il **relâche** le verrou.

```
public final void java.lang.Object.notify()
```

- ▶ Réveille un des threads en attente sur l'objet verrouillé.
- ▶ Le thread réveillé doit **acquérir** le verrou.

```
public final void java.lang.Object.notifyAll()
```

- ▶ Pareil mais réveille tous les threads en attente sur l'objet verrouillé.

Point vocabulaire

- ▶ Un thread demandant l'accès à un code déjà verrouillé est **bloqué**.
- ▶ Un thread qui est mis en pause par l'opération wait est **en attente**.

Implémentation d'un sémaphore

```
class Semaphore {  
    private int counter;  
  
    public Semaphore( int counter ) {  
        this.counter = counter;  
    }  
  
    synchronized public void acquire()  
    throws InterruptedException {  
        while(counter <= 0) wait();  
        --counter;  
    }  
  
    synchronized public void release() {  
        ++counter;  
        notify();  
    }  
}
```

Syntaxes alternatives

En Java : `java.util.concurrent.locks.Condition`

```
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition();

lock.lock();
condition.signal();
condition.await();
lock.unlock();
```

En C++ : `std::condition_variable`

```
mutex mtx;
condition_variable condition;

unique_lock<mutex> lock(mtx);
condition.notify_one();
condition.wait(lock);
// unlock on ~unique_lock
```

Partie C. Correction d'un programme multi-threads

VI. Spécification d'un problème concurrent

1. Problèmes de vivacité
2. Sûreté et vivacité d'un programme

VII. Modèles de mémoire

Que fait ce programme ?

```
public record Friend(String nom) {

    public synchronized void salute(Friend friend) {
        System.out.println(nom + " salutes " + friend.nom);
        friend.answer(this);
    }

    public synchronized void answer(Friend friend) {
        System.out.println(nom + " answers to " + friend.nom);
    }

    public static void main(String[] args) {
        var alphonse = new Friend("Alphonse");
        var gaston = new Friend("Gaston");
        var tA = new Thread(() -> alphonse.salute(gaston));
        var tG = new Thread(() -> gaston.salute(alfonse));
        tA.start();
        tG.start();
    }
}
```

 Exemple inspiré du tutorial Java d'Oracle (Java Documentation)

 Exemples-Java/snippets.liveness.Friends

Exemple d'exécution

```
public synchronized void salute(Friend friend) {  
    System.out.println(nom + " salutes " + friend.nom);  
    friend.answer(this);  
}  
  
public synchronized void answer(Friend friend) {  
    System.out.println(nom + " answers to " + friend.nom);  
}
```



À ce point :

- ▶ t_A est bloqué par t_G sur le verrou de `gaston`
- ▶ t_G est bloqué par t_A sur le verrou de `alphonse`

Interblocage circulaire

Deadlock

Situation d'**interblocage circulaire** où des threads attendent entre eux la libération de ressources avant de continuer leur exécution.



Une solution pour éviter les deadlocks

Composition

Des deadlocks peuvent se produire quand plusieurs verrous sont utilisés.

Théorème

- ▶ Supposons qu'il y a un ordre sur les verrous $v_1 \leq v_2 \leq \dots \leq v_n$.
- ▶ Supposons qu'aucun thread ne cherche à obtenir v alors qu'il possède déjà v' avec $v \leq v'$.
- ▶ Alors l'exécution ne comporte pas de deadlock.

En TD

Utilisez le théorème ci-dessus pour résoudre le problème du dîner des philosophes

Situation de famine

Starvation

Situation où un ou plusieurs threads n'ont **jamais accès** à la ressource.



En TD

Observez les problèmes de famine dans le problème des toilettes unisexes.

Spécification d'un problème réparti

Sûreté (Safety)

- ▶ Rien de mal ne se produit

$$\forall t. P(t)$$

- ▶ Thread-safe : fonctionne correctement en multi-thread (très imprécis)
- ▶ Cohérence : cohérence séquentielle

Vivacité (Liveness)

- ▶ Quelque chose de bien finira par se produire

$$\exists t. P(t)$$

Exemple : programme qui calcule π

Sûreté : À tout instant, un préfixe de l'écriture décimale de π est affiché.

Vivacité : Pour tout n , au moins n chiffres finiront par être affichés.

Sûreté et vivacité

Infirmes ou confirmez les affirmations suivante

1. Tous les empires sont des agglomérats de royaumes.
2. Tous les empires finissent par s'effondrer.

Propriété de sûreté ou de vivacité ?

1. Si deux voitures attendent à une intersection, l'une va finir par passer.
2. Il n'y aura pas d'accident à l'intersection.
3. Le feu va passer au vert.
4. Le feu va passer au vert dans les cinq prochaines minutes.
5. Seules deux choses sont certaines : la mort et les impôts.

Condition de vivacité

Progrès global

Au moins un thread progresse dans son exécution.

- ▶ propriété violée lors d'un interblocage circulaire.

Progrès local

Tous les threads progressent dans leur exécution.

- ▶ propriété violée lors d'une famine.

Condition de vivacité

Progrès global

Au moins un thread progresse dans son exécution.

- ▶ propriété violée lors d'un interblocage circulaire.

Progrès local

Tous les threads progressent dans leur exécution.

- ▶ propriété violée lors d'une famine.

Équité (fairness) des verrous

- ▶ Les blocks `synchronized`, les verrous `ReentrantLock` et les `mutex` ne garantissent que le progrès global.
- ▶ En Java, le constructeur de `ReentrantLock` a un argument optionnel :

```
public ReentrantLock(boolean fair)
```

Caractérisation du progrès global

Deux définitions équivalentes si le nombre de threads est fini

Un objet O vérifie le progrès global si, pour toute exécution :

1. Pour tout suffixe de l'exécution, il existe un thread dont toutes les opérations sur O se terminent.
2. Si l'exécution contient un nombre fini d'opérations sur O , elles se terminent toutes.

Idée de démonstration : 1 $\implies$ 2

Par récurrence sur le nombre de threads dans l'exécution :

1. S'il n'y a qu'un seul thread, toutes ses opérations se terminent
2. S'il y a $n + 1$ threads, l'un d'entre eux termine toutes ses opérations, et on se retrouve avec un suffixe avec n threads.

Idée de démonstration : 2 $\implies$ 1

Par l'absurde : si aucun thread ne terminait l'une de ses opérations, il y aurait un nombre fini d'opérations dans l'exécution, donc elles termineraient toutes.

Partie C. Correction d'un programme multi-threads

VI. Spécification d'un problème concurrent

VII. Modèles de mémoire

1. Problèmes de cohérence
2. La relation happens-before
3. Exemples et contre-exemples
4. Le mot-clé volatile
5. Registres atomiques et instructions spéciales

Que fait ce programme ?

```
class Counter {  
    private int value=0;  
    public synchronized void increment() {  
        ++value;  
    }  
    public synchronized int get() {  
        return value;  
    }  
  
    public static void main(String[] args) {  
        var counter = new Counter();  
  
        var tWriter = new Thread( counter::increment );  
  
        var tReader = new Thread(() -> {  
            while( counter.get()==0 );  
            System.out.println("done");  
        });  
  
        tReader.start();  
        Thread.sleep(250);  
        tWriter.start();  
  
    }  
}
```

 Exemple inspiré de Thibault Delor (InvalidCodeBlog)

 Exemples-Java/snippets.memory.Volatile

Ce code peut-il lever une erreur ?

```
class LongWrites {  
    static long value = -1L;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            value++;  
        }).start();  
        new Thread(() -> {  
            if( value > 0 ) {  
                throw new RuntimeException("reachable?");  
            }  
        }).start();  
    }  
}
```

Ce code peut-il lever une erreur ?

```
class LongWrites {
    static long value = -1L;
    public static void main(String args[]) {
        new Thread(() -> {
            value++;
        }).start();
        new Thread(() -> {
            if( value > 0 ) {
                throw new RuntimeException("reachable?");
            }
        }).start();
    }
}
```

$-1 = 0x\text{FFFFFFFF_FFFFFFFF}$

$0 = 0x\text{00 00 00 00_00 00 00 00}$

$0 < 0x\text{00 00 00 00_FFFFFFFF}$

Spécification du langage Java (JLS §17.7)

“ Implementations of the Java Virtual Machine are *encouraged* to avoid splitting 64-bit values where possible. Programmers are *encouraged* to declare shared 64-bit values as *volatile or synchronize* their programs correctly to avoid possible complications.

”

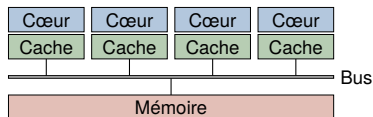
Que peut-on observer ?

```
class Caches {  
    static int x = 0, y = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            x = 1;  
            System.out.println( y );  
        }).start();  
        new Thread(() -> {  
            y = 1;  
            System.out.println( x );  
        }).start();  
    }  
}
```

Que peut-on observer ?

```
class Caches {  
    static int x = 0, y = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            x = 1;  
            System.out.println( y );  
        }).start();  
        new Thread(() -> {  
            y = 1;  
            System.out.println( x );  
        }).start();  
    }  
}
```

Architectures UMA (Uniform Memory Access)



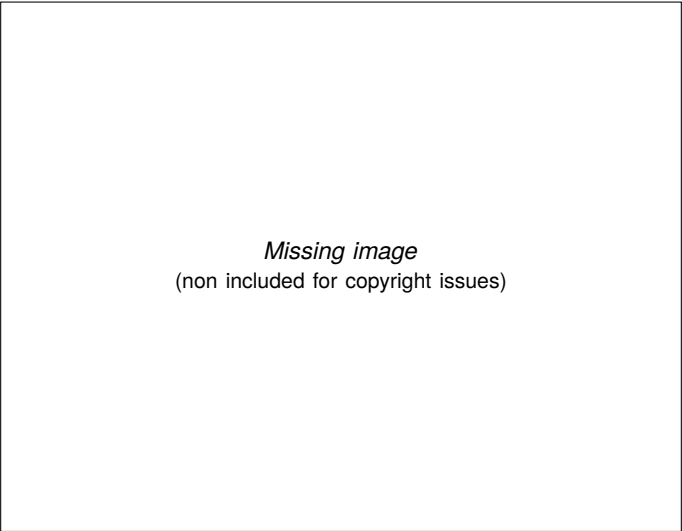
Mot-clé `volatile int x;` (depuis Java 5)

- ▶ Tout le cache est invalidé avant chaque lecture de `x`
- ▶ Tout le cache est invalidé après chaque écriture de `x`
- ▶ Le cache n'est pas invalidé pendant une lecture ou écriture de `x`

Attention

- ▶ Une invalidation de cache est très inefficace !

Échelles de temps



Missing image
(non included for copyright issues)

Quels affichages sont possibles ?

```
class Optimizations {  
    static int sum = 0;  
    static int i = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            for(i=1; i<2; i++) sum+=i;  
        }).start();  
        new Thread(() -> {  
            if(sum != 0) System.out.println(i);  
        }).start();  
    }  
}
```

Quels affichages sont possibles ?

```
class Optimizations {  
    static int sum = 0;  
    static int i = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            for(i=1; i<2; i++) sum+=i;  
        }).start();  
        new Thread(() -> {  
            if(sum != 0) System.out.println(i);  
        }).start();  
    }  
}
```

Compilateur et JVM : respect de la sémantique séquentielle

```
for(i=1; i<2; i++)  
    sum+=i;
```

Quels affichages sont possibles ?

```
class Optimizations {  
    static int sum = 0;  
    static int i = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            for(i=1; i<2; i++) sum+=i;  
        }).start();  
        new Thread(() -> {  
            if(sum != 0) System.out.println(i);  
        }).start();  
    }  
}
```

Compilateur et JVM : respect de la sémantique séquentielle

```
for(i=1; i<2; i++)  
    sum+=i;
```

```
i=1;  
sum=1;  
i=2;
```

Quels affichages sont possibles ?

```
class Optimizations {  
    static int sum = 0;  
    static int i = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            for(i=1; i<2; i++) sum+=i;  
        }).start();  
        new Thread(() -> {  
            if(sum != 0) System.out.println(i);  
        }).start();  
    }  
}
```

Compilateur et JVM : respect de la sémantique séquentielle

```
for(i=1; i<2; i++)  
sum+=i;
```

```
i=1;  
sum=1;  
i=2;
```

```
sum=1;  
i=2;
```

Quels affichages sont possibles ?

```
class Optimizations {  
    static int sum = 0;  
    static int i = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            for(i=1; i<2; i++) sum+=i;  
        }).start();  
        new Thread(() -> {  
            if(sum != 0) System.out.println(i);  
        }).start();  
    }  
}
```

Compilateur et JVM : respect de la sémantique séquentielle

```
for(i=1; i<2; i++)  
    sum+=i;
```

```
i=1;  
sum=1;  
i=2;
```

```
sum=1;  
i=2;
```

Mot-clé `volatile int x;`

- ▶ Pas d'optimisation qui change la position d'une opération sur x

Modèle de Mémoire du langage Java (JMM)

Rappel : Si l'exécution est **séquentiellement cohérente** (SC), l'ensemble des exécutions est décrit par une sémantique d'entrelacement

Problème

- ▶ Les programmes Java **ne sont pas** SC par défaut
- ▶ Comment raisonner sur un programme concurrent ?

Modèle de Mémoire du langage Java (JMM)

Rappel : Si l'exécution est **séquentiellement cohérente** (SC), l'ensemble des exécutions est décrit par une sémantique d'entrelacement

Problème

- ▶ Les programmes Java **ne sont pas** SC par défaut
- ▶ Comment raisonner sur un programme concurrent ?
- ▶ Un programme est **correctement synchronisé** si aucune de ses exécutions SC ne contient de **data race**
- ▶ Tout programme correctement synchronisé est SC

“ *When a program contains two conflicting accesses that are not ordered by a happens-before relationship, it is said to contain a **data race**.*

Spécification du langage Java (JLS §17.4) ”

Modèle de Mémoire du langage Java (JMM)

Rappel : Si l'exécution est **séquentiellement cohérente** (SC), l'ensemble des exécutions est décrit par une sémantique d'entrelacement

Problème

- ▶ Les programmes Java **ne sont pas** SC par défaut
- ▶ Comment raisonner sur un programme concurrent ?
- ▶ Un programme est **correctement synchronisé** si aucune de ses exécutions SC ne contient de **data race**
- ▶ Tout programme correctement synchronisé est SC

“ When a program contains two conflicting accesses that are not ordered by a **happens-before** relationship, it is said to contain a **data race**. ”

Spécification du langage Java (JLS §17.4)

Rappel : la relation **happens-before** est un ordre partiel formé de :

- ▶ un **ordre de programme** fait le lien avec la sémantique séquentielle
- ▶ une **relation synchronises-with** fait le lien entre les threads

Relation synchronizes-with

Définition – Relation synchronizes-with

Certaines actions de synchronisation induisent des dépendances sémantiques capturées par la relation synchronizes-with, notée $\xrightarrow{SW}$:

► Cycle de vie d'un thread :

- $t.start() \xrightarrow{SW}$ première action de t
- $t.interrupt() \xrightarrow{SW}$ détection de l'interruption par t
- dernière action de $t \xrightarrow{SW}$ détection de la fin de t
- initialisation d'une variable par défaut $\xrightarrow{SW}$ première action de tout thread

► Blocs `synchronized(o)` sur le même moniteur `o` :

- sortie d'un bloc $\xrightarrow{SW}$ entrées ultérieures réussies dans un bloc

► Accès à la même variable x déclarée `volatile` :

- écriture dans $x \xrightarrow{SW}$ lectures dans x après cette écriture et avant la suivante

Relation synchronizes-with

Définition – Relation synchronizes-with

Certaines actions de synchronisation induisent des dépendances sémantiques capturées par la relation synchronizes-with, notée $\xrightarrow{SW}$:

► Cycle de vie d'un thread :

- $t.start() \xrightarrow{SW}$ première action de t
- $t.interrupt() \xrightarrow{SW}$ détection de l'interruption par t
- dernière action de $t \xrightarrow{SW}$ détection de la fin de t
- initialisation d'une variable par défaut $\xrightarrow{SW}$ première action de tout thread

► Blocs `synchronized(o)` sur le même moniteur `o` :

- sortie d'un bloc $\xrightarrow{SW}$ entrées ultérieures réussies dans un bloc

► Accès à la même variable `x` déclarée `volatile` :

- écriture dans `x` $\xrightarrow{SW}$ lectures dans `x` après cette écriture et avant la suivante

Remarque : définition de avant et après

- Les relations temporelles sont bien définies si l'exécution est SC.
- Sinon, la JLS introduit un ordre total de synchronisation sur ces actions.

Relation happens-before

Définition – Ordre de programme

- ▶ Pour chaque thread t_i , la **sémantique intra-thread** (opérationnelle à petits pas) définit un **ordre total** $\xrightarrow{i}_{po}$ entre tous les pas de t_i
- ▶ L'**ordre de programme** est l'**ordre partiel** union $\xrightarrow{po} = \bigcup_{t_i} \xrightarrow{i}_{po}$

Relation happens-before

Définition – Ordre de programme

- Pour chaque thread t_i , la **sémantique intra-thread** (opérationnelle à petits pas) définit un **ordre total** $\xrightarrow{po}_i$ entre tous les pas de t_i
- L'**ordre de programme** est l'**ordre partiel** union $\xrightarrow{po} = \bigcup_i \xrightarrow{po}_i$

Définition – Relation happens-before

- L'**ordre partiel strict** $\xrightarrow{hb}$ est la fermeture transitive de $\xrightarrow{sw} \cup \xrightarrow{po}$:
 - S'il existe un thread t_i tel que $e \xrightarrow{po}_i e'$, alors $e \xrightarrow{hb} e'$
 - Si e et e' sont des actions de synchronisation et $e \xrightarrow{sw} e'$, alors $e \xrightarrow{hb} e'$
 - Si $e \xrightarrow{hb} e_1 \xrightarrow{hb} e_2 \xrightarrow{hb} \dots \xrightarrow{hb} e'$, alors $e \xrightarrow{hb} e'$
- Les actions e et e' sont **concurrentes** ($e \parallel e'$) si $e \not\xrightarrow{hb} e'$ et $e' \not\xrightarrow{hb} e$.

Cohérence happens-before

Définition – Data race

Une **data race** est un couple d'actions $\langle e, e' \rangle$ telles que :

- ▶ Les deux actions sont concurrentes selon $\xrightarrow{hb} : e || e'$
- ▶ Les deux actions accèdent à la même variable non-volatile
- ▶ Au moins une action est une écriture

Définition – Programme correctement synchronisé

- ▶ Une exécution est **correctement synchronisée** si elle n'a pas de data race
- ▶ Un programme est **correctement synchronisé** si toutes ses exécutions **séquentiellement cohérentes** sont correctement synchronisées
 - ▶ **Remarque** : Il suffit donc de chercher les data races sur des exécutions SC

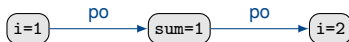
Garantie de la spécification – Cohérence happens-before

- ▶ Tout programme **correctement synchronisé** est SC
- ▶ Sinon, **aucune garantie** : bug !

Retour sur l'exemple du compilateur

```
class Optimizations {  
    static int sum = 0;  
    static int i = 0;  
    public static void main(String args[]) {  
        new Thread(() -> {  
            for(i=1; i<2; i++) sum+=i;  
        }).start();  
        new Thread(() -> {  
            if( sum != 0 ) System.out.println( i );  
        }).start();  
    }  
}
```

Happens Before

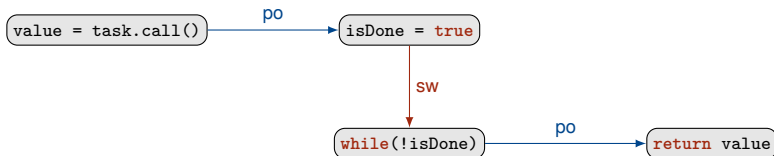


- ▶ Il y a des data races sur `sum` et sur `i`
- ▶ Les deux variables doivent être volatiles

Exemple (schématique)

```
class Future<T> {  
  
    final Callable<T> task;  
    volatile boolean isDone = false;  
    T value = null;  
  
    void init() { // par thread du thread pool  
        value = task.call();  
        isDone = true;  
    }  
  
    T get() { // par autre thread  
        while( !isDone );  
        return value;  
    }  
}
```

Happens Before

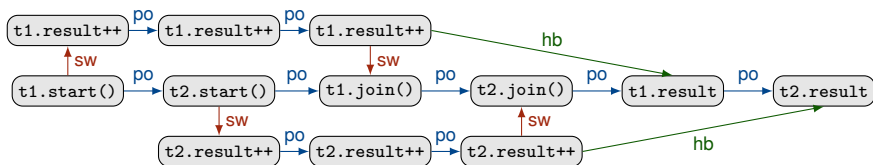


Retour sur le calcul de π

```

public class PiMonteCarlo implements Runnable {
    private int result = 0;
    public void run() {
        for(int i = 0; i<nbIterations; i++) {
            double x = random.nextDouble(); double y = random.nextDouble();
            if(x*x + y*y < 1) result++;
        }
    }
    static int experiment() {
        for(int i = 0; i<nbThreads; i++){
            tasks[i] = new PiMonteCarlo(); threads[i] = new Thread(tasks[i]);
        }
        for(var t : threads) t.start(); for(var t : threads) t.join();
        int result = 0;
        for(var t : tasks) result+=t.result;
        return result;
    }
}

```



Retour sur l'exemple introductif

```
class Counter {  
    private int value=0;  
    public synchronized void increment() {  
        ++value;  
    }  
    public int get() {  
        return value;  
    }  
  
    public static void main(String[] args) {  
        var counter = new Counter();  
        var tWriter = new Thread( counter::increment );  
        var tReader = new Thread(() -> {  
            while( counter.get()==0 );  
            System.out.println("done");  
        });  
        tReader.start();  
        Thread.sleep(250);  
        tWriter.start();  
    }  
}
```

- ▶ Data race entre les lectures par `tReader` et l'écriture par `tWriter`
- ▶ Il faut protéger la lecture, soit par `volatile`, soit par `synchronized`

Le compteur bloquant

Si beaucoup de lectures

```
class Counter {  
    private volatile int value=0;  
    public synchronized void increment() {  
        ++value;  
    }  
    public int get() {  
        return value;  
    }  
}
```

Si plus d'incrémentations que de lectures

```
class Counter {  
    private int value=0;  
    public synchronized void increment() {  
        ++value;  
    }  
    public synchronized int get() {  
        return value;  
    }  
}
```

Comment éviter les data races ?

Règle d'or pour le langage Java

Pour chaque variable x , il faut s'assurer que x est dans l'un des cas suivants :

- ▶ x est déclarée **final**
 - ▶ Adoptez un style de programmation fonctionnelle
- ▶ x n'est accessible que par un seul thread
 - ▶ Si x est un membre d'une classe C , s'applique à toutes les instances de C
- ▶ x est déclarée **volatile**
 - ▶ Utile pour la communication simple entre threads
- ▶ x est **toujours** accédée dans un bloc **synchronized** sur le même moniteur
 - ▶ Mais attention à ne pas trop limiter le parallélisme
- ▶ x n'est pas sujet à une data race
 - ▶ Prouvez-le en commentaire

Modèle de mémoire en C++

Le mot-clé `volatile` en C++

- ▶ Limite certaines optimisations locales du compilateur
- ▶ Ne gère pas la synchronisation entre threads

Barrières de mémoire en C++

- ▶ `void std::atomic_thread_fence( std::memory_order order )`
- ▶ Choix des arcs ajoutés à l'ordre $\xrightarrow{\text{SW}}$: read/write, write/read...

“ *There are only two hard things in Computer Science :
cache invalidation and naming things.*

”

Phil Karlton

C++ Core Guidelines

- ▶ CP.8 : Don't try to use `volatile` for synchronization
- ▶ Utiliser `template <class T> struct std::atomic`

 Tutoriel de Ori Lahav sur les modèles de mémoire en C++

Limite de `volatile` : les tableaux

Tableaux volatiles

```
volatile int array[];  
array = new int[10];           // volatile  
array[5] = 3;                  // pas volatile
```

Limite de `volatile` : les tableaux

Tableaux volatiles

```
volatile int array[];  
array = new int[10];           // volatile  
array[5] = 3;                  // pas volatile
```

Première solution

```
class VolatileInt{public volatile int x=0;}  
VolatileInt array[] = new VolatileInt[10];  
array[5] = new VolatileInt(); // pas volatile  
array[5].x = 3;               // volatile
```

Limite de `volatile` : les tableaux

Tableaux volatiles

```
volatile int array[];  
array = new int[10];           // volatile  
array[5] = 3;                  // pas volatile
```

Première solution

```
class VolatileInt{public volatile int x=0;}  
VolatileInt array[] = new VolatileInt[10];  
array[5] = new VolatileInt(); // pas volatile  
array[5].x = 3;               // volatile
```

Meilleure solution : `java.util.concurrent.atomic.AtomicIntegerArray`

```
AtomicIntegerArray array = new AtomicIntegerArray(10);  
array.set(5, 3);          // volatile
```

Registres atomiques

Java

- ▶ Dans le package `java.util.concurrent.atomic`
 - ▶ `AtomicInteger`, `AtomicLong`, `AtomicReference<T>`, ...

C++

- ▶ Structure `template <class T> struct std::atomic`
- ▶ Fonctions de `<atomic>`

Méthodes `get()` et `set()` (Java) ou `load()` et `store()` (C++)

- ▶ Lectures et écritures atomiques

Méthodes RMW (read-modify-write)

- ▶ test-and-set, fetch-and-add, compare_and_swap...
- ▶ Opérations atomiques matérielles (instructions spéciales)
- ▶ Permettent la synchronisation non-bloquante

Structures de données non-bloquante en Java

- ▶ **Compteur** : `java.util.concurrent.atomic.LongAdder`
 - ▶ `void add(long x)` : incrémente
 - ▶ `long sum()` : lit
- ▶ **File** : `java.util.concurrent.ConcurrentLinkedQueue<E>` :
 - ▶ `void add(E e)` : ajoute en queue
 - ▶ `E peek()` : lit la tête
 - ▶ `E poll()` : lit et supprime la tête
- ▶ **Liste doublement chaînée** : `java.util.concurrent.ConcurrentLinkedDeque<E>`
 - ▶ `void addFirst(E e), E getFirst(), E pollFirst()` : accède la tête
 - ▶ `void addLast(E e), E getLast(), E pollLast()` : accède la queue
- ▶ **Table de hashage** : `java.util.concurrent.ConcurrentHashMap<K,V>`
 - ▶ `put(K key, V value)` : ajoute
 - ▶ `get(K key)` : lit
 - ▶ `remove(K key)` : supprime une clé
 - ▶ `putIfAbsent(K key, V value)` : ajoute ou retourne la valeur

Ces structures de données seront étudiées au deuxième semestre

Partie D. Algorithmes d'Exclusion Mutuelle

VIII. Exclusion mutuelle à partir de read/write

1. Algorithme de Peterson
2. Algorithme de la Boulangerie de Lamport

IX. Utilisation d'instructions spéciales

X. Conclusion

Spécification du problème

Interface du verrou

```
interface Lock {  
    void lock() ;  
    void unlock() ;  
}
```

Spécification logique

Exclusion mutuelle : Il ne peut jamais y avoir deux threads en section critique en même temps

Starvation-freedom : Si un thread cherche à entrer en section critique, il finira par entrer en section critique

Un premier essai

```
class NaiveLock1 implements Lock {  
    private volatile boolean taken = false;  
    public void lock() {  
        while( taken );  
        taken = true;  
    }  
    public void unlock() {  
        taken = false;  
    }  
}
```

Exercice

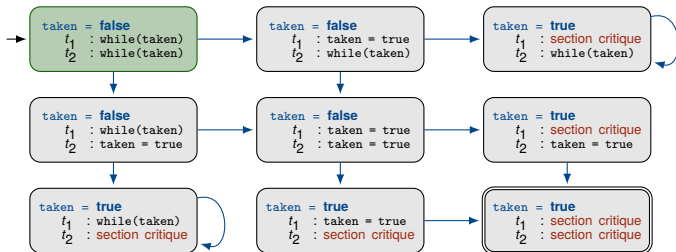
Deux threads, t_1 et t_2 , veulent entrer concurremment en section critique.

- ▶ Tracer la machine à états du système.
- ▶ Montrer que l'exclusion mutuelle n'est pas respectée
- ▶ Montrer que la starvation-freedom n'est pas respectée

Un premier essai

```
class NaiveLock1 implements Lock {
    private volatile boolean taken = false;
    public void lock() {
        while( taken );
        taken = true;
    }
    public void unlock() {
        taken = false;
    }
}
```

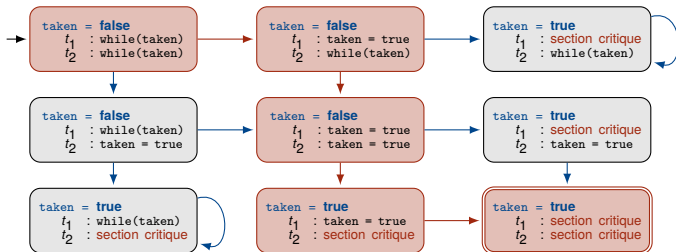
Exercice



Un premier essai

```
class NaiveLock1 implements Lock {
    private volatile boolean taken = false;
    public void lock() {
        while( taken );
        taken = true;
    }
    public void unlock() {
        taken = false;
    }
}
```

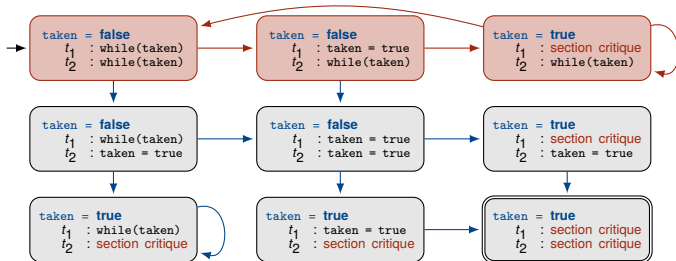
Exercice



Un premier essai

```
class NaiveLock1 implements Lock {
    private volatile boolean taken = false;
    public void lock() {
        while( taken );
        taken = true;
    }
    public void unlock() {
        taken = false;
    }
}
```

Exercice



Un deuxième essai

On suppose deux processus, d'identifiants 0 et 1.

```
class NaiveLock2 implements Lock {  
    private final AtomicIntegerArray entering = new AtomicIntegerArray(2)  
  
    public void lock() {  
        int i = ThreadID.get();  
        entering.set(i, 1);  
  
        while( entering.get(1-i) == 1 );  
    }  
    public void unlock() {  
        int i = ThreadID.get();  
        entering.set(i, 0);  
    }  
}
```

Exercice

- ▶ Cet algorithme respecte-t-il l'exclusion mutuelle ?
- ▶ Cet algorithme est-il starvation-free ?

Un troisième essai

On suppose deux processus, d'identifiants 0 et 1.

```
class NaiveLock3 implements Lock {  
  
    private volatile int priority = 0;  
    public void lock() {  
        int i = ThreadID.get();  
  
        while( priority != i );  
    }  
    public void unlock() {  
        int i = ThreadID.get();  
        priority = 1-i;  
    }  
}
```

Exercice

- ▶ Cet algorithme respecte-t-il l'exclusion mutuelle ?
- ▶ Cet algorithme est-il starvation-free ?

Algorithme de Peterson

On suppose deux processus, d'identifiants 0 et 1.

```
class PetersonLock implements Lock {  
    private final AtomicIntegerArray entering = new AtomicIntegerArray(2);  
    private volatile int priority = 0;  
    public void lock() {  
        int i = ThreadID.get();  
        entering.set(i, 1);  
        priority = 1-i;  
        while( entering.get(1-i) == 1 && priority != i );  
    }  
    public void unlock() {  
        int i = ThreadID.get();  
        entering.set(i, 0);  
    }  
}
```

Exercice

- ▶ Cet algorithme respecte-t-il l'exclusion mutuelle ?
- ▶ Cet algorithme est-il starvation-free ?

 G. L. Peterson. *Myths About the Mutual Exclusion Problem*. IPL (1981)

 Exemples-Java/snippets.mutualexclusion.PetersonLock

Algorithme simplifié

shared variables:

$_ tickets[n] \leftarrow [\infty, \dots, \infty]$

operation lock():

```
1   $tickets[i] \leftarrow 0$   
2  let  $t_i$  such that  $\langle t_i, i \rangle > \max_{j: tickets[j] < \infty} \langle tickets[j], j \rangle$   
3   $tickets[i] \leftarrow t_i$   
4  wait until  $\langle t_i, i \rangle = \min_j \langle tickets[j], j \rangle$ .
```

operation unlock():

$_ tickets[i] \leftarrow \infty$.



Algorithme simplifié

shared variables:

$tickets[n] \leftarrow [\infty, \dots, \infty]$

operation lock():

```
1  tickets[i] ← 0
2  let ti such that  $\langle t_i, i \rangle > \max_{j: tickets[j] < \infty} \langle tickets[j], j \rangle$ 
3  tickets[i] ← ti
4  wait until  $\langle t_i, i \rangle = \min_j \langle tickets[j], j \rangle$ .
```

operation unlock():

$tickets[i] \leftarrow \infty$.



Algorithme simplifié

shared variables:

$_ tickets[n] \leftarrow [\infty, \dots, \infty]$

operation lock():

```
1  tickets[i] ← 0
2  let  $t_i$  such that  $\langle t_i, i \rangle > \max_{j: tickets[j] < \infty} \langle tickets[j], j \rangle$ 
3  tickets[i] ←  $t_i$ 
4  wait until  $\langle t_i, i \rangle = \min_j \langle tickets[j], j \rangle$ .
```

operation unlock():

$_ tickets[i] \leftarrow \infty$.



Algorithme simplifié

shared variables:

$_ tickets[n] \leftarrow [\infty, \dots, \infty]$

operation lock():

```
1   $tickets[i] \leftarrow 0$   
2  let  $t_i$  such that  $\langle t_i, i \rangle > \max_{j: tickets[j] < \infty} \langle tickets[j], j \rangle$   
3   $tickets[i] \leftarrow t_i$   
4  wait until  $\langle t_i, i \rangle = \min_j \langle tickets[j], j \rangle$ .
```

operation unlock():

$_ tickets[i] \leftarrow \infty$.



Algorithme simplifié

shared variables:

$\lfloor \text{tickets}[n] \leftarrow [\infty, \dots, \infty]$

operation lock():

```

1   $\text{tickets}[i] \leftarrow 0$ 
2  let  $t_i$  such that  $\langle t_i, i \rangle > \max_{j: \text{tickets}[j] < \infty} \langle \text{tickets}[j], j \rangle$ 
3   $\text{tickets}[i] \leftarrow t_i$ 
4  wait until  $\langle t_i, i \rangle = \min_j \langle \text{tickets}[j], j \rangle$ .
```

operation unlock():

$\lfloor \text{tickets}[i] \leftarrow \infty$.



Algorithme simplifié

shared variables:

$\lfloor \text{tickets}[n] \leftarrow [\infty, \dots, \infty]$

operation lock():

```

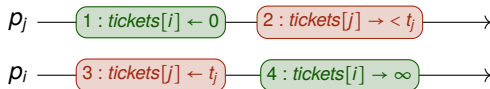
1   $\text{tickets}[i] \leftarrow 0$ 
2  let  $t_i$  such that  $\langle t_i, i \rangle > \max_{j: \text{tickets}[j] < \infty} \langle \text{tickets}[j], j \rangle$ 
3   $\text{tickets}[i] \leftarrow t_i$ 
4  wait until  $\langle t_i, i \rangle = \min_j \langle \text{tickets}[j], j \rangle$ .
```

operation unlock():

$\lfloor \text{tickets}[i] \leftarrow \infty$.

Exclusion mutuelle :

Soient p_i et p_j en section critique, avec $\langle t_i, i \rangle < \langle t_j, j \rangle$



Algorithme de Lamport

```

class BakeryLock implements Lock {
    private final AtomicIntegerArray entering = new AtomicIntegerArray(n);
    private final AtomicIntegerArray priority = new AtomicIntegerArray(n);
    public void lock() {
        int i = ThreadID.get();
        entering.set(i, 1);
        priority.set(i, 1 + max(priority));
        entering.set(i, 0);
        for(int j = 0; j < n; j++) {
            while(true) {
                if(entering.get(j) == 1) continue;
                int pI = priority.get(i);
                int pJ = priority.get(j);
                if(pJ == 0) break;
                if(pJ > pI) break;
                if(pJ == pI && j >= i) break;
            }
        }
        public void unlock() {
            priority.set(ThreadID.get(), 0);
        }
    }
}

```

 L. Lamport. *A new solution of Dijkstra's concurrent programming problem*. CACM (1974)

 Exemples-Java/snippets.mutualexclusion.BakeryLock

Partie D. Algorithmes d'Exclusion Mutuelle

VIII. Exclusion mutuelle à partir de read/write

IX. Utilisation d'instructions spéciales

1. Algorithme spin-lock

X. Conclusion

L'algorithme spin-lock

```
class SpinLock {  
    private AtomicBoolean taken = new AtomicBoolean(false);  
  
    public void lock() {  
        while( taken.getAndSet(true) );  
    }  
  
    public void unlock() {  
        taken.set(false);  
    }  
}
```

L'opération atomique test-and-set

```
getAndSet(T val) {  
    T old = get();  
    set(val);  
    return old;  
}
```

Java : `T getAndSet(T val)`

C++ : `T exchange(T val)`

 E. W. Dijkstra. *Over de sequentialiteit van procesbeschrijvingen (About the sequentiality of process descriptions)* (1962-1963)

 `Exemples-Java/snippets/mutualexclusion.SpinLock`

Partie D. Algorithmes d'Exclusion Mutuelle

VIII. Exclusion mutuelle à partir de read/write

IX. Utilisation d'instructions spéciales

X. Conclusion

1. Bilan
2. Perspectives

Bilan

Calcul réparti en mémoire partagée

- ▶ La concurrence crée de nouveaux problèmes de sûreté
- ▶ La synchronisation pour les résoudre limite la vivacité

Attention aux contre-intuitions !

- ▶ `x++` ne signifie pas « incrémenter `x` »
- ▶ La cohérence séquentielle n'est pas donnée par défaut

Limiter le partage des données

- ▶ Utiliser des objets immutables
- ▶ Ne pas partager si possible

Un monde en constante évolution

Modèles à passage de messages

- ▶ Cloud computing
- ▶ Réseaux pair-à-pair

Algorithmique du réparti en calcul réparti

- ▶ Problèmes de calculabilité
- ▶ Problèmes de complexité

Modèles de programmation concurrente

- ▶ Loom project et Java 21
 - ▶ Threads virtuels
 - ▶ Concurrency structurée
- ▶ Modèles de concurrence de Go, Gleam, Rust...

Mémoire transactionnelle

Mémoire transactionnelle logicielle

```
void increment (Register<Integer> X) {  
    Transaction t = new STMTransaction();  
    while (!t.isCommitted()) {  
        try {  
            t.begin();  
            X.write(t, X.read(t) + 1);  
            t.try_to_commit();  
        } catch (AbortException e) {}  
    }  
}
```

Mémoire transactionnelle

Mémoire transactionnelle logicielle

```
void increment (Register<Integer> X) {  
    Transaction t = new STMTransaction();  
    while (!t.isCommitted()) {  
        try {  
            t.begin();  
            X.write(t, X.read(t) + 1);  
            t.try_to_commit();  
        } catch (AbortException e) {}  
    }  
}
```

TP4

1. Lire l'article sur Madoc
2. Implémenter l'algorithme TL2